

OUTPUT: Interest after 7 Years = \$8137.50
Total Amount after 7 Years = \$20637.50

RESULT:
The program has been executed successfully.

EX NO: 2	Finding the Range of Statistical Data
Date:	

AIM:

Pinaccio went to the school. His Maths teacher gave the task to Pinaccio that to find the range from the largest number and smallest number in the statistical data. Can you help him to find the range?

Algorithm:

Step 1: Start the program.

Step 2: Read the largest number and the smallest number from the user.

Step 3: Calculate the range by subtracting the smallest number from the largest number.

$\text{Range} = \text{greatest integer}(G) - \text{smallest integer}(S)$

Step 4: Display the calculated range.

Step 5: Stop the program.

CODE:

```
G = int(input())
S = int(input())
print(f"The Range of given data = {G-S}")
```

Test Case 1

INPUT(STDIN): 10
 18

OUTPUT: The Range of given data = 8

Test Case 2

INPUT(STDIN): 16
 5

OUTPUT: The Range of given data = 11

RESULT:

The program has been executed successfully.

EX NO: 3	Electricity Bill Calculation Using Power Operation
Date:	

AIM:

The Electricity Officer has mentioned the total counts of unit and amount. The officer informs the customer the bill amount in a unique format. The format given by electricity officer as follow: But customers are finding the difficult to find the exact amount that needs to be paid. Can you help the customers?

Algorithm:

Step 1: Start the program.

Step 2: Read the total units consumed and the cost per unit from the user.

Step 3: Calculate the total bill amount by raising the units consumed to the power of the cost per unit.

$\text{Total Bill Amount} = \text{unitconsumed} ^ \text{costperunit}$

Step 4: Display the calculated electricity bill amount.

Step 5: Stop the program.

CODE:

```
a = int(input())
b = int(input())
print(a**b)
```

Test Case 1

INPUT(STDIN): 23
 3

OUTPUT: 12167

Test Case 2

INPUT(STDIN): 19
 5

OUTPUT: 2476099

RESULT:

The program has been executed successfully.

EX NO: 4	Calculating Surface Area of a Cuboid
Date:	

AIM:

Selvan was playing with an object of random size for stress relief. Selvan knows that the Length, Width, and Height of the object. But he would like to know the surface area of the object he is playing with. Can you help him in finding it?

Algorithm:

Step 1: Start the program.

Step 2: Read the length, width, and height of the object from the user.

Step 3: Calculate the surface area using the formula:

$$\text{Surface Area} = 2 \times (\text{length} \times \text{width} + \text{width} \times \text{height} + \text{height} \times \text{length})$$

Step 4: Display the calculated surface area of the object.

Step 5: Stop the program.

CODE:

```
length = int(input())
width = int(input())
height = int(input())
print(2*(length*width+width*height+height*length))
```

Test Case 1

INPUT(STDIN): 3
 2
 3

OUTPUT: 42

Test Case 2

INPUT(STDIN): 3
 7
 4

OUTPUT: 122

RESULT:

The program has been executed successfully.

EX NO: 5	Calculating Area of a Room
Date:	

AIM:

Arif planned to make a room cleaning robot for his college mini project competition. First, he has to code program to simulate the robot movements inside the room. He measured the length and width of the room. Once the values are available, his program should compute and display the area of the room. Can you help Arif with a suitable logic for the code?

Algorithm:

Step 1: Start the program.

Step 2: Read the length and width of the room from the user.

Step 3: Calculate the area of the room by multiplying the length and width.

Step 4: Display the calculated area of the room up to two decimal places.

Step 5: Stop the program.

CODE:

```
length = float(input())
width = float(input())
print(f'{length*width:.2f}')
```

Test Case 1

INPUT(STDIN): 23.44
 22.33

OUTPUT: 523.42

Test Case 2

INPUT(STDIN): 27.42
 29.35

OUTPUT: 804.78

RESULT:

The program has been executed successfully.

EX NO: 6	Restaurant Bill Calculation with Tax and Tip
Date:	

AIM:

Aaron took his girlfriend Binita to a restaurant as he got a job of his dreams. Since he had small welcome interview he was little bit tensed. Binita figured this out and to get back Aaron's confidence she gave him a little task, When they received the bill for the food they ordered, She asked Aaron to find out the tax amount of the bill and tip for the meal through a computer code. Aaron can use your local tax rate when computing the amount of tax owing. Can you help Aaron to code the suitable logic.

Algorithm:

- Step 1:** Start the program.
- Step 2:** Read the total bill amount from the user.
- Step 3:** Calculate the tax amount using the local tax rate.
- Step 4:** Calculate the tip amount for the meal.
- Step 5:** Calculate the total bill amount by adding the bill amount, tax, and tip.
- Step 6:** Display the tax amount, tip amount, and the total bill amount.
- Step 7:** Stop the program.

CODE:

```
amount = int(input())
local_Tax = 18*amount/100
tip_Amount = 5*amount/100
print(f"The Tax is {local_Tax:.2f}")
print(f"The Tip is {tip_Amount:.2f}")
print(f"Total Bill With Tax and Tip is {amount+local_Tax+tip_Amount:.2f}")
```

Test Case 1

INPUT(STDIN): 521

OUTPUT: The Tax is 93.78

 The Tip is 26.05

 Total Bill With Tax and Tip is 640.83

Test Case 2

INPUT(STDIN): 734

OUTPUT: The Tax is 132.12

The Tip is 36.70

Total Bill With Tax and Tip is 902.82

RESULT:

The program has been executed successfully.

EX NO: 7	Calculating Distance Between Two Points
Date:	

AIM:

Janaki wants to find the distance between the two points (x1,y1) and (x2,y2). She knows the formula for the distance is $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$. Can you help her to create a program for finding the distance?

Algorithm:

Step 1: Start the program.

Step 2: Read the coordinates of the two points (x_1, y_1) and (x_2, y_2) from the user.

Step 3: Calculate the distance using the formula:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Step 4: Display the calculated distance between the two points.

Step 5: Stop the program.

CODE:

```
import math
x1 = int(input())
y1 = int(input())
x2 = int(input())
y2 = int(input())
distance = ((x2-x1)**2)+((y2-y1)**2)
print(f" {math.sqrt(distance)}")
```

Test Case 1

```
INPUT(STDIN):  1
                1
                3
                3
OUTPUT:        2.8284271247461903
```

Test Case 2

```
INPUT(STDIN):  0
                1
                2
                2
OUTPUT:        2.23606797749979
```

RESULT:

The program has been executed successfully.

OUTPUT: The area of triangle is 10.825317547305483

RESULT:

The program has been executed successfully.

EX NO: 9	Conversion of Time into Seconds
Date:	

AIM:

Vetrivel wants to convert the units of time into seconds. He gets the days, hours, minutes, seconds from the user. Can you help him to convert the units in to seconds.

Algorithm:

Step 1: Start the program.

Step 2: Read the number of days, hours, minutes, and seconds from the user.

Step 3: Convert all the given time units into seconds and calculate the total number of seconds.

Step 4: Display the total time in seconds.

Step 5: Stop the program.

CODE:

```

days = int(input())
hours = int(input())
minutes = int(input())
seconds = int(input())
print((days*24*60*60)+(hours*3600)+(minutes*60)+seconds)

```

Test Case 1

```

INPUT(STDIN):    2
                  1
                  1
                  1
OUTPUT:          176461

```

Test Case 2

```

INPUT(STDIN):    3
                  2
                  1
                  1
OUTPUT:          266461

```

RESULT:

The program has been executed successfully.

EX NO: 10	Calculating Area of a Circle
Date:	

AIM:

In geometry, the area enclosed by a circle of radius r is πr^2 . Here the Greek letter π represents a constant, approximately equal to 3.142, which is equal to the ratio of the circumference of any circle to its diameter. Subash wants to find the area of circle for the given radius. Can you help him to calculate the area of circle for the given radius?

Algorithm:

Step 1: Start the program.

Step 2: Read the radius of the circle from the user.

Step 3: Calculate the area of the circle using the formula:

$$\text{Area} = \pi \times r^2$$

Step 4: Display the calculated area of the circle.

Step 5: Stop the program.

CODE:

```
r = float(input())
area = 3.142*r*r
print(f"The area of the circle with radius {r} is: {area}")
```

Test Case 1

INPUT(STDIN): 1.1

OUTPUT: The area of the circle with radius 1.1 is: 3.8018200000000006

Test Case 2

INPUT(STDIN): 2.2

OUTPUT: The area of the circle with radius 2.0 is: 12.568

RESULT:

The program has been executed successfully.

EX NO: 11	Profit and Loss Determination
Date:	

AIM:

Aarav a newbie entrepreneur was studying the profit and loss of his company. He found out for some products cost price is greater than selling price, there was some loss and for other products he got some profit Can you kindly automate this small work for him by creating a code that checks what arav wants?

Algorithm:

Step 1: Start the program.

Step 2: Read the cost price and selling price of the product from the user.

Step 3: Compare the cost price and selling price.

- If cost price is greater than selling price, display “**Loss**”.
- If cost price is less than selling price, display “**Profit**”.
- Otherwise, display “**No Profit No Loss**”.

Step 4: Stop the program.

CODE:

```
costPrice = int(input())
sellingPrice = int(input())
if costPrice > sellingPrice:
    print("Loss")
elif costPrice < sellingPrice:
    print("Profit")
else:
    print("No Profit No Loss")
```

Test Case 1

INPUT(STDIN): 2389
 3114

OUTPUT: Profit

Test Case 2

INPUT(STDIN): 2985
 1463

OUTPUT: Loss

RESULT:

The program has been executed successfully.

EX NO: 12	Check Whether the Currency Value is Even or Odd
Date:	

AIM:

The Paytm announced a Cashback offer for the people of Tamil Nadu which is a onetime offer for the new year. But to avail the Cashback users need to pass the simple tasks given by Paytm. One such task given by Paytm is to check the nature of the currency value provided by Paytm.

Algorithm:

Step 1: Start the program.

Step 2: Read the currency value from the user.

Step 3: Check whether the currency value is divisible by 2.

If the currency value is divisible by 2, display **“Even Currency”**.

Otherwise, display **“Odd Currency”**.

Step 4: Stop the program.

CODE:

```
Currency = int(input())
if Currency%2 == 0:
    print("Even Currency")
else:
    print("Odd Currency")
```

Test Case 1

INPUT(STDIN): 1985

OUTPUT: Odd Currency

Test Case 2

INPUT(STDIN): 1642

OUTPUT: Even Currency

RESULT:

The program has been executed successfully.

EX NO: 13	Sum or Difference of Two Numbers
Date:	

AIM: Caleb and Salima are living in interior village of Nilgais. Since government of Tamil Nadu announced lockdown both of them struck in the village and it's been very hard for them to spend their day because of the lack of friends in the village. So, they planned to play a technical game on Lockdown days.

The rule of the game is simple: When one among Caleb and Salima say two numbers to the other. The person at the receiving end need to tell the difference between the numbers if the first number is greater than the second number, otherwise they have to tell the sum of those two numbers.

Algorithm:

Step 1: Start the program.

Step 2: Read two numbers from the user.

Step 3: Compare the two numbers.

- If the first number is greater than the second number, calculate and display their difference.
- Otherwise, calculate and display their sum.

Step 7: Stop the program.

CODE:

```
num1 = int(input())
num2 = int(input())
if num1>num2:
    print(num1-num2)
else:
    print(num1+num2)
```

Test Case 1

INPUT(STDIN): 1746
 1429

OUTPUT: 317

Test Case 2

INPUT(STDIN): 1921
 3238

OUTPUT: 5159

RESULT:

The program has been executed successfully.

EX NO: 14	Apple Budget Check
Date:	

AIM:

Caleb and Irfan are purchasing apples which were priced according to their size. But their budget is minimum. So, they plan to choose one small, one medium and one large apple so that it will fit in their budget. So can you help them choose the right apple by creating a logic by naming three apples they choose as apple1, apple2, apple3. Then check the condition if apple2 is greater than apple1 and apple3 is greater than apple2.

Algorithm:

Step 1: Start the program.

Step 2: Read the prices of three apples: apple1, apple2, and apple3 from the user.

Step 3: Check whether the price of apple2 is greater than apple1 and the price of apple3 is greater than apple2.

- If both conditions are satisfied, display **“Fit into Budget”**.
- Otherwise, display **“Doesn't fit into Budget”**.

Step 7: Stop the program.

CODE:

```
apple1 = int(input())
apple2 = int(input())
apple3 = int(input())
if apple2>apple1 and apple3>apple2:
    print("Fit into Budget")
else:
    print("Doesn't fit into Budget")
```

Test Case 1

INPUT(STDIN): 380
 170
 429

OUTPUT: Doesn't fit into Budget

Test Case 2

INPUT(STDIN): 102
 237
 398

OUTPUT: Fit into Budget

RESULT:

The program has been executed successfully.

EX NO: 15	Speed Difference Between Two Racers
Date:	

AIM:

Arav and Aaron are participating in the Bike racing. Arav have crossed some mile stores earlier and Aaron crossed some mile stores earlier during their racing, because they have changed their speeds at different times. Both of them like to know the difference in speeds between them at different stages of racing. Can you help finding the speed difference between Arav and Aaron?

Algorithm:

Step 1: Start the program.

Step 2: Read the speeds of Arav and Aaron from the user.

Step 3: Compare the two speeds.

- ☐ If Arav's speed is greater than Aaron's speed, calculate and display the difference.
- ☐ Otherwise, calculate and display the difference by subtracting Arav's speed from Aaron's speed.

Step 4: Stop the program.

CODE:

```
arav = int(input())
aaron = int(input())
if arav > aaron:
    print(arav - aaron)
else:
    print(aaron - arav)
```

Test Case 1

INPUT(STDIN): 74
 51
OUTPUT: 23

Test Case 2

INPUT(STDIN): 76
 89
OUTPUT: 13

RESULT:

The program has been executed successfully.

EX NO: 16	Student Grade Calculation
Date:	

AIM:

Mrs. Swathy wants students to calculate their **percentage and grade** based on the marks obtained in five subjects. This program computes the percentage and displays the corresponding grade according to the given criteria.

Algorithm:

Step 1: Start the program.

Step 2: Read the marks obtained in five subjects from the user.

Step 3: Calculate the percentage by finding the average of the five subject marks.

Step 4: Display the calculated percentage.

Step 5: Determine the grade based on the percentage:

- If percentage is 90 or above, display Grade A.
- If percentage is 80 or above, display Grade B.
- If percentage is 70 or above, display Grade C.
- If percentage is 60 or above, display Grade D.
- If percentage is 40 or above, display Grade E.
- Otherwise, display Grade F.

Step 6: Stop the program.

CODE:

```

mark1 = int(input())
mark2 = int(input())
mark3 = int(input())
mark4 = int(input())
mark5 = int(input())
per = (mark1+mark2+mark3+mark4+mark5)/5
print(f"{per:.2f} Percent")
if per >= 90:
    print("Grade A")
elif per >= 80:
    print("Grade B")
elif per >= 70:

```

```
        print("Grade C")
elif per>=60:
    print("Grade D")
elif per>=40:
    print("Grade E")
else:
    print("Grade F")
```

Test Case 1

INPUT(STDIN):	55
	45
	35
	65
	75
OUTPUT:	55.00 Percent
	Grade E

Test Case 2

INPUT(STDIN):	89
	96
	56
	98
	95
OUTPUT:	86.80 Percent
	Grade B

RESULT:

The program has been executed successfully.

EX NO: 17	Leap Year Check
Date:	

AIM:

Laasya wants to check whether a given **birth year is a leap year or not**. This program automates the task by applying the leap year rules and displays the appropriate result.

Algorithm:

Step 1: Start the program.

Step 2: Read the year from the user.

Step 3: Check the leap year conditions:

- If the year is divisible by 400, it is a leap year.
- If the year is divisible by 4 and not divisible by 100, it is a leap year.

Step 4:

- If any of the above conditions are satisfied, display "LEAP YEAR".
- Otherwise, display "NOT A LEAP YEAR".

Step 5: Stop the program.

CODE:

```
year = int(input())
if year%400 == 0 or (year%4==0 and year %100!=0):
    print("LEAP YEAR")
else:
    print("NOT A LEAP YEAR")
```

Test Case 1

INPUT(STDIN): 2004

OUTPUT: LEAP YEAR

Test Case 2

INPUT(STDIN): 1994

OUTPUT: NOT A LEAP YEAR

RESULT:

The program has been executed successfully.

EX NO: 18	ATM Withdrawal Validation and Balance Update
Date:	

AIM:

Atifa wants to withdraw money from an ATM. The withdrawal is allowed only if the requested amount is a multiple of 5 and the account balance is sufficient to cover the withdrawal along with the bank charge. This program validates the withdrawal request and displays the current balance or an error message accordingly.

Algorithm:

Step 1: Start the program.

Step 2: Read the withdrawal amount and the initial account balance from the user.

Step 3: Check whether the withdrawal amount is a multiple of 5 and whether the initial balance is sufficient to cover the withdrawal amount along with the bank charge.

Step 4:

- If the conditions are satisfied:
 - Deduct the withdrawal amount and bank charge from the initial balance.
 - Display the current balance after the successful transaction.
 - Display the initial balance.
- Otherwise:
 - Display "Invalid Withdrawal Request".
 - Display the initial balance.

Step 5: Stop the program.

CODE:

```

withdrawal_amount = int(input())
initial_amount = float(input())
if initial_amount > withdrawal_amount and withdrawal_amount % 5 == 0:
    print(f"Current Balance : {initial_amount - withdrawal_amount - 0.5:.2f}")
    print(f"Initial Balance : {initial_amount:.2f}")
else:
    print("Invalid Withdrawal Request")
    print(f"Initial Balance : {initial_amount:.2f}")

```

Test Case 1

INPUT(STDIN): 23500
 67300.50

OUTPUT: Current Balance : 43800.00
 Initial Balance : 67300.50

Test Case 2

INPUT(STDIN): 21650
 19600.00

OUTPUT: Invalid Withdrawal Request
 Initial Balance : 19600.00

RESULT:

The program has been executed successfully.

EX NO: 19	Simple Calculator Using Operator
Date:	

AIM:

Arulmozhivarman plans to conduct a quiz where participants are given two numbers and an operator. This program performs the corresponding arithmetic operation based on the given operator and displays the result.

Algorithm:

Step 1: Start the program.

Step 2: Read the operator and two numbers from the user.

Step 3: Check the operator and perform the corresponding operation:

- If the operator is +, add the two numbers.
- If the operator is -, subtract the second number from the first.
- If the operator is *, multiply the two numbers.
- If the operator is /, divide the first number by the second.

Step 4:

- Display the calculated result.
- If the operator is not valid, display "Invalid Input".

Step 5: Stop the program.

CODE:

```
oprator = input()
n1 = float(input())
n2 = float(input())
if oprator == '+':
    print(f'{n1+n2}')
elif oprator == '-':
    print(f'{n1-n2}')
elif oprator == '*':
    print(f'{n1*n2:.2f}')
elif oprator == '/':
    print(f'{n1/n2}')
else:
    print("Invalid Input")
```

Test Case 1

INPUT(STDIN): *
 77.6
 54.9

OUTPUT: 4260.24

Test Case 2

INPUT(STDIN): \$
 98.1
 14.3

OUTPUT: Invalid Input

RESULT:

The program has been executed successfully.

EX NO: 20	Room Rent Estimation Portal
Date:	

AIM:

Aadi and Tara want to estimate the total room rent for their stay while traveling. Since room rents increase by 20% during peak seasons (April and May), this program calculates and displays the total rent based on the month, room rent per day, and number of days of stay.

Algorithm:

Step 1: Start the program.

Step 2: Read the month, room rent per day, and total number of days of stay from the user.

Step 3: Check whether the given month is April or May.

Step 4:

- If the month is April or May, calculate the total rent by adding 20% to the room rent.
- Otherwise, calculate the total rent without any additional charge.

Step 5: Display the total room rent to be paid.

Step 6: Stop the program.

CODE:

```
month = int(input())
room_rent = float(input())
days = int(input())
if month == 4 or month == 5:
    total = room_rent * days * 1.20
else:
    total = room_rent * days
print(f"Rs. {total:.2f}")
```

Test Case 1

INPUT(STDIN): 7
 4650.00
 11

OUTPUT: Rs.51150.00

Test Case 2

INPUT(STDIN): 4
 3800.00
 9

OUTPUT: Rs.41040.00

RESULT:

The program has been executed successfully.

EX NO: 21	Sum of Multiples of 3 or 5
Date:	

AIM:

Euler asked Ramanujan to find the sum of all natural numbers below a given limit that are multiples of 3 or 5. This program reads the upper limit from the user and calculates the required sum.

Algorithm:

Step 1: Start the program.

Step 2: Read the upper limit of the natural number and store it in variable N.

Step 3: Initialize a variable to store the sum as zero.

Step 4: Repeat the following steps for all numbers from 3 to $N - 1$:

- Check if the number is divisible by 3 or 5.
- If true, add the number to the sum.

Step 5: Display the calculated sum of multiples of 3 or 5.

Step 6: Stop the program.

CODE:

```
num = int(input())
sum=0
for i in range(3,num):
    if i%3==0 or i%5==0:
        sum+=i
print(sum)
```

Test Case 1

INPUT(STDIN): 10

OUTPUT: 23

Test Case 2

INPUT(STDIN): 15

OUTPUT: 45

RESULT:

The program has been executed successfully.

EX NO: 22	Separate Even and Odd Numbers
Date:	

AIM:

Thanos and Iron Man were given a challenge by Hulk to separate even and odd numbers from a given set of numbers. This program reads a list of numbers and displays the even numbers and odd numbers separately.

Algorithm:

Step 1: Start the program.

Step 2: Read the total number of elements to be entered.

Step 3: Initialize two empty lists to store even numbers and odd numbers.

Step 4: For each number entered by the user:

- Check whether the number is divisible by 2.
- If it is divisible by 2, store it in the even list.
- Otherwise, store it in the odd list.

Step 5: Display all the even numbers.

Step 6: Display all the odd numbers.

Step 7: Stop the program.

CODE:

```
size = int(input())
even = []
odd = []
for i in range(1,size+1):
    num = int(input())
    if num%2==0:
        even.append(num)
    else:
        odd.append(num)
for i in even:
    print(i,end=" ")
print()
for i in odd:
    print(i,end=" ")
```

Test Case 1

INPUT(STDIN): 3
 22
 50
 11

OUTPUT: 22 50
 11

Test Case 2

INPUT(STDIN): 5
 2
 4
 6
 5
 1

OUTPUT: 2 4 6
 5 1

RESULT:

The program has been executed successfully.

EX NO: 23	Matrix Creation and Display
Date:	

AIM:

To create and display a matrix by reading the number of rows and columns along with the matrix elements from the user. This program arranges the given elements into a matrix format and displays them row-wise.

Algorithm:

Step 1: Start the program.

Step 2: Read the number of rows and columns of the matrix from the user.

Step 3: Read all the matrix elements from the user and store them in a list.

Step 4: Initialize an index variable to access elements from the list.

Step 5: Using nested loops, display the elements in matrix form according to the given number of rows and columns.

Step 6: Stop the program.

CODE:

```

row = int(input())
column = int(input())
li = []
for i in range(row*column):
    li.append(int(input()))
idx = 0
for i in range(row):
    for j in range(column):
        print(li[idx],end=" ")
        idx+=1
    print()

```

Test Case 1

INPUT(STDIN): 3
 3
 1
 2
 3
 4
 5
 6
 7

8
9
OUTPUT: 1 2 3
4 5 6
7 8 9

Test Case 2

INPUT(STDIN): 2
2
1
2
3
4

OUTPUT: 1 2
3 4

RESULT:

The program has been executed successfully.

EX NO: 24	Sorting and Displaying Character Names
Date:	

AIM:

Marvel Studios wants to select character names for a new movie to make it a blockbuster. This program reads a list of character names, sorts them in alphabetical order, and displays the sorted list.

Algorithm:

Step 1: Start the program.

Step 2: Read the total number of character names to be entered.

Step 3: Read all the character names from the user and store them in a list.

Step 4: Sort the list of names in alphabetical order.

Step 5: Display the sorted list of character names one by one.

Step 6: Stop the program.

CODE:

```
n = int(input())
names = []
for i in range(n):
    names.append(input())
names.sort()
for name in names:
    print(name)
```

Test Case 1

INPUT(STDIN): 4
 Spider-Man
 Iron Man
 The Hulk
 Thor

OUTPUT: Iron Man
 Spider-man
 The Hulk
 Thor

Test Case 2

INPUT(STDIN): 3
 Thanos
 Wolverine
 Iron Man

OUTPUT: Iron Man
 Thanos
 Wolverine

RESULT:

The program has been executed successfully.

EX NO: 25	Generation of Pentagonal Numbers
Date:	

AIM:

Pentagonal numbers are generated by the formula, $P_{\{n\}} = n(3n - 1) / 2$ Generate the first n pentagonal numbers.

Algorithm:

Step 1: Start the program.

Step 2: Read the number of terms n from the user.

Step 3: Repeat the following steps for values of i from 1 to $n - 1$:

- Calculate the pentagonal number using the formula

$$P_i = \frac{i(3i-1)}{2}$$

- Display the calculated pentagonal number.

Step 4: Stop the program.

CODE:

```
test_case = int(input())
for i in range(1,test_case):
    print(int(i*(3*i-1)/2))
```

Test Case 1

INPUT(STDIN): 10

OUTPUT: 1
5
12
22
35
51
70
92
117

Test Case 2

INPUT(STDIN): 5

OUTPUT: 1
5
12
22

RESULT:

The program has been executed successfully.

EX NO: 26	Triangular Flower Decoration Pattern
Date:	

AIM:

The Matriculation school wants to display a triangular flower decoration pattern for its Annual Day function. Roses are placed on the boundary of the triangle and tulips are placed inside. This program displays the final decoration layout based on the given number of rows.

Algorithm:

Step 1: Start the program.

Step 2: Read the number of rows for the decoration from the user.

Step 3: For each row from 1 to the given number of rows, repeat the following:

- For each position in the row, check whether it lies on the boundary of the triangle.
- If it is on the boundary (first row, last row, first column, or diagonal), display 1 (Rose).
- Otherwise, display 0 (Tulip).

Step 4: Move to the next line after completing each row.

Step 5: Stop the program.

CODE:

```
num = int(input())
for i in range(1,num+1):
    for j in range(1,i+1):
        if i==1 or i==num or j==1 or j==i:
            print(1, end=" ")
        else:
            print(0,end=" ")
    print()
```

Test Case 1

INPUT(STDIN): 7

OUTPUT: 1
1 1
1 0 1
1 0 0 1
1 0 0 0 1
1 0 0 0 0 1
1 0 0 0 0 0 1

Test Case 2

INPUT(STDIN): 9

OUTPUT: 1
1 1
1 0 1
1 0 0 1
1 0 0 0 1
1 0 0 0 0 1
1 0 0 0 0 0 1
1 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 1

RESULT:

The program has been executed successfully.

EX NO: 27	Descending Seat Layout Display
Date:	

AIM:

The event management team wants to display the seating layout for a concert where families are seated based on the number of tickets booked. Families with more members are seated in the front rows, and those with fewer members are seated in the back rows. This program displays the seating pattern in descending order based on the given number of rows.

Algorithm:

Step 1: Start the program.

Step 2: Read the total number of rows from the user.

Step 3: For each row from the given number down to 1, repeat the following:

- Print the row number repeatedly equal to the number of seats in that row.

Step 4: Move to the next line after completing each row.

Step 5: Stop the program.

CODE:

```
num = int(input())
for i in range(num,0,-1):
    for j in range(1,i+1):
        print(i, end=" ")
    print()
```

Test Case 1

```
INPUT(STDIN):    11
OUTPUT:          11 11 11 11 11 11 11 11 11 11 11
                  10 10 10 10 10 10 10 10 10 10
                  9 9 9 9 9 9 9 9
                  8 8 8 8 8 8 8
                  7 7 7 7 7 7
                  6 6 6 6 6
                  5 5 5 5
                  4 4 4
                  3 3 3
                  2 2
                  1
```


Test Case 2

INPUT(STDIN): 8

OUTPUT: 8 8 8 8 8 8 8 8
7 7 7 7 7 7 7
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1

RESULT:

The program has been executed successfully.

EX NO: 28	Seating Arrangement for Magic Show
Date:	

AIM:

The show organizers want to create a seating arrangement layout for a magic show where couples and singles are seated based on seat numbering rules. This program displays the seating layout row-wise when the total number of rows is provided.

Algorithm:

Step 1: Start the program.

Step 2: Read the total number of rows for the seating arrangement from the user.

Step 3: For each row from 1 to the given number of rows, repeat the following:

- Print the row number repeatedly equal to the row count to represent the seating layout.

Step 4: Move to the next line after completing each row.

Step 5: Stop the program.

CODE:

```
num = int(input())
for i in range(1,num+1):
    for j in range(1,i+1):
        print(i,end=" ")
    print()
```

Test Case 1

INPUT(STDIN): 9

OUTPUT:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9
```

Test Case 2

INPUT(STDIN): 15

OUTPUT: 1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9
10 10 10 10 10 10 10 10 10 10
11 11 11 11 11 11 11 11 11 11 11
12 12 12 12 12 12 12 12 12 12 12 12
13 13 13 13 13 13 13 13 13 13 13 13 13
14 14 14 14 14 14 14 14 14 14 14 14 14 14
15 15 15 15 15 15 15 15 15 15 15 15 15 15 15

RESULT:

The program has been executed successfully.

EX NO: 29	Sum of Digits of Factorial
Date:	

AIM:

To calculate the sum of the digits in the factorial of a given number. This program computes the factorial of the number and then finds the sum of its digits.

Algorithm:

Step 1: Start the program.

Step 2: Read the number n from the user.

Step 3: Calculate the factorial of the given number using a loop.

Step 4: Convert the factorial value into digits and calculate the sum of those digits.

Step 5: Display the calculated sum of the digits.

Step 6: Stop the program.

CODE:

```
num = int(input())
fact = 1
for i in range(1,num+1):
    fact*=i
sum = 0
for i in str(fact):
    sum+=int(i)
print(sum)
```

Test Case 1

INPUT(STDIN): 10

OUTPUT: 27

Test Case 2

INPUT(STDIN): 100

OUTPUT: 648

RESULT:

The program has been executed successfully.

EX NO: 30	Sum of Even-Valued Fibonacci Terms
Date:	

AIM:

To find the sum of even-valued terms in the Fibonacci sequence. This program generates Fibonacci numbers up to a given number of terms and calculates the sum of the even terms.

Algorithm:

Step 1: Start the program.

Step 2: Read the number of terms and store it in the variable nterms.

Step 3: Check whether the number of terms is greater than 1.

- If not, display an appropriate message and stop.

Step 4: Initialize the first two Fibonacci numbers and variables to track the count and the sum of even-valued terms.

Step 5: Generate the Fibonacci sequence up to the given number of terms:

- Calculate the next term as the sum of the previous two terms.
- If the term is even, add it to the even sum.
- Update the previous terms and increment the count.

Step 6: Display the sum of even-valued Fibonacci terms.

Step 7: Stop the program.

CODE:

```
terms = int(input())
if terms <= 1:
    print("Please enter a positive integer greater than 1")
else:
    a = 1
    b = 2
    count = 2
    even_sum = 2
    while count < terms:
        c = a + b
        if c % 2 == 0:
            even_sum += c
        a = b
```

```
b = c  
count += 1  
print(even_sum)
```

Test Case 1

INPUT(STDIN): 2

OUTPUT: 2

Test Case 2

INPUT(STDIN): -1

OUTPUT: Please enter a positive integer greater than 1

RESULT:

The program has been executed successfully.

EX NO: 31	Change Lowercase letters to uppercase and uppercase letters to lowercase
DATE :	

AIM

To develop a Python program that reads a string from the user and converts each character's case changing lowercase letters to uppercase and uppercase letters to lowercase.

ALGORITHM

Step 1: Start the program.

Step 2: Read a string input from the user and store it in the variable ch.

Step 3: Initialize an empty string variable result.

Step 4: For each character c in the string ch, perform the following:

 If c is a lowercase letter, convert it to uppercase and append it to result.

 Else, convert it to lowercase and append it to result.

Step 5: Display the final converted string result.

Step 6: End the program.

CODE

```
ch=str(input())
result = ""
for c in ch:
    if c.islower():
        result += c.upper()
    else:
        result += c.lower()
print(result)
```

Test Case 1

Input (STDIN)	maHenDraSinghDhOni
Expected Output	MAhENdRAsINGHdHoNI

Test Case 2

Input (STDIN)	uMeshYaDhaV
Expected Output	UmESHyAdHAv

RESULT

Thus, the Python program to convert lowercase letters into uppercase and uppercase letters into lowercase was successfully executed and verified.

EX NO: 32	Demonstrating Concatenation, Repetition, and Substring Membership Testing
DATE:	

AIM

To write a Python program to perform string operations such as concatenation, repetition, and checking the presence of one string inside another.

ALGORITHM

- Step 1:** Start the program.
- Step 2:** Read the first string from the user and store it in str1.
- Step 3:** Read the second string from the user and store it in str2.
- Step 4:** Display the concatenation of str1 and str2.
- Step 5:** Display str1 repeated two times.
- Step 6:** Display str2 repeated two times.
- Step 7:** Assign str1 to the variable string and str2 to the variable sub_string.
- Step 8:** Check whether sub_string is present in string and display the result.
- Step 9:** Check whether string is present in sub_string and display the result.
- Step 10:** Check whether sub_string is not present in string and display the result.
- Step 11:** Check whether string is not present in sub_string and display the result.
- Step 12:** Stop the program.

CODE

```
str1=input()
str2=input()
print(str1 + str2)
print(str1 * 2)
print(str2 * 2)
string = str1
sub_string = str2
print(string.find(sub_string) != -1)
```

```

print(sub_string.find(string) != -1)

print(string.find(sub_string) == -1)

print(sub_string.find(string) == -1)

```

Test Case 1

Input (STDIN)	eLab Auto Evaluation eLab
Expected Output	eLab Auto EvaluationeLab eLab Auto EvaluationeLab Auto Evaluation eLabeLab True False False True

Test Case 2

Input (STDIN)	eLab Tool
Expected Output	UmESHyAdHAv eLabTool eLabeLab ToolTool False False True True

RESULT

Thus, the Python program to perform string concatenation, string repetition, and substring presence checking was successfully executed and verified.

EX NO: 33	Calculate the Arithmetic Series Sum Based on Character Frequency in a Binary String
DATE :	

AIM

To write a Python program that reads multiple test cases, counts the number of 1s in a given binary string.

ALGORITHM

Step 1: Start the program.

Step 2: Read an integer t representing the number of test cases.

Step 3: Repeat the following steps t times:

Step 3.1: Read an integer l (length of the string).

Step 3.2: Read the binary string s .

Step 3.3: Count the number of '1' characters in s and store it in the variable $ones$.

Step 3.4: Compute $result = ones \times (ones + 1) / 2$.

Step 3.5: Round the value of $result$ and store it in

r . **Step 3.6:** Print the value of r .

Step 4: Stop the program.

CODE

```
import math

t = int(input())

for _ in range(t):

    l = int(input())

    s = str(input())
```

```
ones = s.count('1')
```

```
result = ones * (ones + 1) / 2
```

```
r = round(result)
```

```
print(r)
```

Test Case 1

Input (STDIN)	3
	6
	111011
	8
	0110111
	5
	10011
Excepted Output	15
	21
	6

Test Case 2

Input (STDIN)	5
	3
	101
	6
	011101
	8

	11100101
	4
	0010
	7
	1000011
Expected Output	3
	10
	15
	1
	6

RESULT

Thus, the Python program to count the number of ones in a binary string and compute the value using the given mathematical formula was successfully executed and verified.

EX NO: 34	Calculate the Minimum and Maximum Possible Hamming Distance Between Two Strings with Wildcards
DATE :	

AIM

To write a Python program that compares two strings containing characters and the symbol to find the minimum and maximum number of mismatches possible.

ALGORITHM

Step 1: Start the program.

Step 2: Read an integer t representing the number of test cases.

Step 3: Repeat the following steps for each test case:

Step 3.1: Read string s1.

Step 3.2: Read string s2.

Step 3.3: Initialize two variables mn = 0 and mx = 0.

Step 3.4: For each position i in the strings, perform the following:

Step 3.4.1: If both characters are known (not '?') and different, increment mn by 1.

Step 3.4.2: If any one of the characters is '?', increment mx by 1.

Step 3.4.3: Else if both characters are different, increment mx by 1.

Step 3.5: Print the values of mn and mx.

Step 4: Stop the program.

CODE

```
t = int(input())
for _ in range(t):
    s1=str(input())
    s2=str(input())
    mn = 0
    mx = 0
    for i in range(len(s1)):
```

```

a = s1[i]
b = s2[i]
if a != '?' and b != '?' and a != b:
    mn += 1
if a == '?' or b == '?':
    mx += 1
elif a != b:
    mx += 1
print(mn, mx)

```

Test Case 1

Input (STDIN)	2 w?q?wr ?aqr?r gi?r?y? g?swt?t
---------------	---

Expected Output	0 4 1 6
-----------------	----------------

Test Case 2

Input (STDIN)	5 bha?ht? ?habja? yu?hat? a?fhrew gy?ae?w ghabee? vh??a?q hagt?sq ry?u?wr yyg?a?r
---------------	---

Expected Output	2 5
	3 6
	2 5
	2 6
	1 5

RESULT

Thus, the Python program to calculate the minimum and maximum mismatches between two strings containing wildcard characters was successfully executed and verified.

EX NO: 35	Implementing Parity Checks and Character Exclusions
DATE :	

AIM

To write a Python program to validate a vehicle registration tag based on given conditions of even digit sums and vowel restriction.

ALGORITHM

Step 1: Start the program.

Step 2: Read the vehicle registration tag as input.

Step 3: Define a list of vowels
`v = ['A', 'E', 'I', 'O', 'U', 'Y']`.

Step 4: Check the following conditions:

Step 4.1: Verify that the sum of digits at positions (0,1) is even.

Step 4.2: Verify that the sum of digits at positions (3,4) is even.

Step 4.3: Verify that the sum of digits at positions (4,5) is even.

Step 4.4: Verify that the sum of digits at positions (7,8) is even.

Step 4.5: Check that the character at position 2 is not a vowel.

Step 5: If all the above conditions are satisfied, print "**Allowed**".

Step 6: Else if the character at position 2 is a vowel or any digit pair sum is odd, print "**Arrest**".

Step 7: Otherwise, print "**NILL**".

Step 8: Stop the program.

CODE

```

tag=str(input())

v = ['A', 'E', 'I', 'O', 'U', 'Y']

if (int(tag[0]) + int(tag[1])) % 2 == 0 and \
    (int(tag[3]) + int(tag[4])) % 2 == 0 and \
    (int(tag[4]) + int(tag[5])) % 2 == 0 and \

```

```

(int(tag[7]) + int(tag[8])) % 2 == 0 and \
tag[2] not in v:
    print("Allowed")
elif tag[2] in v or any((int(tag[i]) + int(tag[i+1])) % 2 != 0 for i in [0, 3, 4, 7]):
    print("Arrest")
else:
    print("NIL")

```

Test Case 1

Input (STDIN)	12E345-67
Expected Output	Arrest

Test Case 2

Input (STDIN)	11B242-73
Expected Output	Allowed

RESULT

Thus, the Python program to validate a vehicle registration tag based on even digit sum conditions and vowel restriction was successfully executed and verified.

EX NO: 36	Utilizing Nested Loops and String Slicing Techniques
DATE :	

AIM

To write a Python program to generate and display all possible substrings of a given string.

ALGORITHM

Step 1: Start the program.

Step 2: Read a string from the user and store it in the variable s.

Step 3: Use two nested loops to generate substrings:

Step 3.1: The first loop selects the starting index i.

Step 3.2: The second loop selects the ending index j.

Step 4: Extract the substring from index i to j.

Step 5: Print each substring on a new line.

Step 6: Stop the program.

CODE

```
s = str(input())
for i in range(len(s)):
    for j in range(i + 1, len(s) + 1):
        print(s[i:j])
```

Test Case 1

Input (STDIN)	abc
Expected Output	a ab abc b bc c

Test Case 2

Input (STDIN)	abcd
Expected Output	a ab abc abcd b bc bcd c cd d

RESULT

Thus, the Python program to generate all possible substrings of a given string was successfully executed and verified.

EX NO: 37	String Type Validation in Differentiating using Pure Alphabetic Strings from Numeric and Alphanumeric Inputs
DATE :	

AIM

To write a Python program to determine whether the given input strings are numeric or not using built-in string functions.

ALGORITHM

Step 1: Start the program.

Step 2: Read an integer n representing the number of test cases.

Step 3: Repeat the following steps n times:

Step 3.1: Read the string str1.

Step 3.2: If str1 contains only letters, print **False**.

Step 3.3: If str1 contains only digits, print **True**.

Step 3.4: Otherwise (if the string contains mixed characters), print **True**.

Step 4: Stop the program.

CODE

```
n=int(input())
for i in range(n):
    str1=str(input())
    if str1.isalpha():
        print(False)
    elif str1.isdigit():
        print(True)
    else:
        print(True)
```

Test Case 1

Input (stdin)	1 radha82
Expected Output	True

Test Case 2

Input (STDIN)	2 radha rajiv19
Expected Output	False True

RESULT

Thus, the Python program to check whether given strings are numeric or contain nonalphabetic characters was successfully executed and verified.

EX NO: 38	Vowel Frequency Analysis using Counting with Case Normalization
DATE :	

AIM

To write a Python program to count the number of vowels present in each input string.

ALGORITHM

Step 1: Start the program.

Step 2: Read an integer Keywordcount representing the number of strings.

Step 3: Repeat the following steps for each string:

Step 3.1: Read the string and convert it to lowercase.

Step 3.2: Initialize a vowel counter Keywordvowel = 0.

Step 3.3: Traverse each character in the string.

Step 3.4: If the character is a vowel (a, e, i, o, u), increment the counter.

Step 3.5: Print the total vowel count.

Step 4: Stop the program.

CODE

```
Keywordcount = int(input())
for _ in range(Keywordcount):
    str1=str(input()).lower()
    Keywordvowel = 0
    for char in str1:
        if char in "aeiou":
            Keywordvowel += 1
    print(Keywordvowel)
```

Test Case 1

Input (STDIN)	3 invertible inverse reverse
---------------	---------------------------------------

Expected Output	4 3 3
-----------------	-------------

Test Case 2

Input (STDIN)	4 Geppetto Fairy Pinocchio Teacher
---------------	--

Expected Output	3 2 4 3
-----------------	------------------

RESULT

Thus, the Python program to count the number of vowels in each input string was successfully executed and verified.

EX NO: 39	Binary String Uniformity Analysis Single-Bit Flips via Character Frequency Counts
DATE : 06.03.2026	

AIM

To write a Python program to verify whether a given binary string contains only one occurrence of either 0 or 1.

ALGORITHM

Step 1: Start the program.

Step 2: Read an integer t representing the number of test cases.

Step 3: For each test case, perform the following steps:

Step 3.1: Read the binary string s.

Step 3.2: Count the number of '0' characters in s and store it in the variable zero.

Step 3.3: Count the number of '1' characters in s and store it in the variable one.

Step 3.4: If zero == 1 OR one == 1, print "YES".

Step 3.5: Otherwise, print "NO".

Step 4: Stop the program.

CODE

```
t = int(input())
for _ in range(t):
    s = str(input())
    zero = s.count('0')
    one = s.count('1')
    if zero == 1 or one == 1:
        print("YES")
    else:
        print("NO")
```

Test Case 1

Input (STDIN)	3
	1010
	101
	100
Expected Output	NO
	YES
	YES

Test Case 2

Input (STDIN)	6
	010
	1101
	0011
	101110
	0001
	0010
Expected Output	YES
	YES
	NO
	NO
	YES
	YES

RESULT

Thus, the Python program to check whether a binary string has exactly one 0 or exactly one 1 was successfully executed and verified.

EX NO: 40	Checking the given String is Palindrome or Not
DATE : 06.03.2026	

AIM

To write a Python program to check whether the given string is a palindrome or not.

ALGORITHM

Step 1: Step 1: Start.

Step 2: Define a function check_palindrome().

Step 3: Read a string s from the user using input().

Step 4: Remove leading and trailing spaces using strip().

Step 5: Reverse the string using slicing s[::-1].

Step 6: Compare the original string s with the reversed string.

Step 7: If both strings are equal, print "Palindrome".

Step 8: Otherwise, print "Not Palindrome".

Step 9: Call the function check_palindrome().

Step 10: Stop.

CODE

```
def check_palindrome():
    s = input()
    if s == s[::-1]:
        print("Palindrome")
    else:
        print("Not Palindrome")
check_palindrome()
```

Test Case 1

Input (STDIN)	madam
Expected Output	Palindrome

Test Case 2

Input (stdin)	baa
Expected Output	Not Palindrome

RESULT

Thus, the Python program to determine the game outcome based on the parity of the number of 1s in a binary string was successfully executed and verified.

EX NO: 41	FACTORIAL USING FUNCTIONS
Date: 06.03.2026	

AIM:

Sudan wants to find the factorial of a given number. Can you help him to calculate the factorial of a number (a non-negative integer) using Function. The function accepts the number as an argument.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a function named factorial(n) that accepts a number as an argument.

Step 3: Inside the function, initialize a variable fact = 1.

Step 4: Check if n is equal to **0 or 1**.

- If true, return **1** because factorial of 0 and 1 is 1.

Step 5: Otherwise, use a **loop from 1 to n**.

Step 6: In each iteration multiply the value of fact by the current number.

- $\text{fact} = \text{fact} \times i$

Step 7: After the loop ends, return the value of fact.

Step 8: Take the input number n from the user.

Step 9: Call the function factorial(n).

Step 10: Display the factorial result.

Step 11: Stop the program.

PROGRAM:

```
def fact(n):
    if n == 0: return 1
    else:
        f = 1
        for i in range(1, n + 1): f *= i
    return f
```

```
print(fact(int(input())))
```

TEST CASE 1:

INPUT (STDIN)

4

EXPECTED OUTPUT

24

TEST CASE 2:

INPUT (STDIN)

5

EXPECTED OUTPUT

120

RESULT:

The program has been executed successfully.

